Wildshot Operator Manual

How you run the seven-repo machine — derived digest, snapshot 2026-07-30. Planning docs + Decision Deck win on any conflict.

Status: DERIVED digest, snapshot **2026-07-30**. This manual compiles protocols you already ruled — it invents nothing. On any conflict, the source wins: `docs/16-ECOSYSTEM_MAP.md`, `docs/18-AGENT_SYNC_PROTOCOL.md`, `docs/08-DECISION_REGISTER.md`, the Decision Deck register, and each repo's contract doc. Regenerate this guide after any ruling that changes protocol (the quick card too).

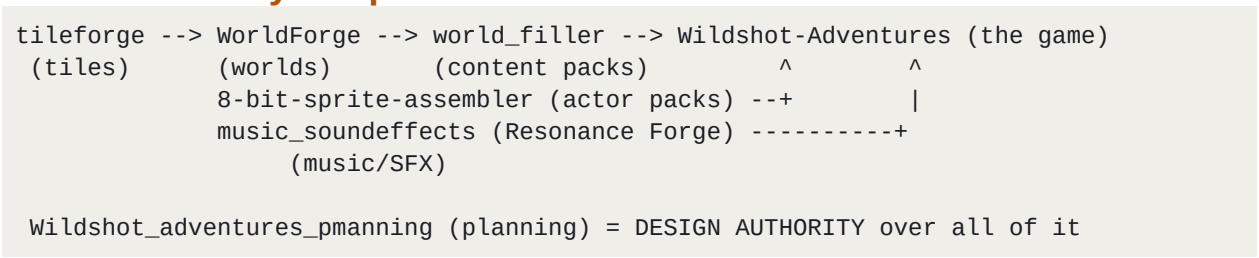
1. What this is

You operate a seven-repo pipeline where AI agents do the implementing and you make every call that matters. The system already has strong laws for what *agents* must do; this manual is the mirror image — what **you, the operator**, do at each part of the machine. It has a companion: `OPERATOR_QUICK_CARD.md`, the one-glance version.

Your job splits into three roles, and almost every task below belongs to exactly one of them:

- **Designer** — rulings, feel verdicts, acceptances. Only you can do these; the whole verdict system exists to protect their integrity.
- **Operator** — running sessions, moving decisions and events through the deck and sync log, keeping repos honest.
- **Producer** — hours, weekly GIF, publishing, tester logistics. The self-accountability layer (PROD-01) that keeps the project shipping.

2. The machine you operate



Universal contract: **frozen versioned pack + validating importer that refuses bad packs + manifest that never changes shape.** Downstream never edits upstream artifacts; problems flow upstream as recorded asks.

What only YOU decide, per repo:

Repo	It makes	Your calls there
planning	decisions, plans, records	every ruling; deck taps; doc acceptances
game	the playable build	feel verdicts, acceptances, adjudications, [T] rates
WorldForge	worlds → game packs	look/walkability verdicts (city walks, holds rounds)
world_filler	content packs	visual verdicts per round; consumption timing (post-Gate-1)

Repo	It makes	Your calls there
tileforge	tile packages	package promotion (e.g. REF3 v2 is user-gated)
assembler	actor/boss sheets	actor mappings (family:variant), sheet drops
Resonance Forge	music/SFX packs	**human listening gate** — automation may reject, never approve

3. Authority — the one-way flow

1. **Planning repo rules everything.** Implementing repos never reinterpret or amend a design decision. Conflicts resolve in planning, never in an implementing repo's commits.
2. **The lane rule:** no repo's session executes another repo's work. Cross-repo needs become recorded asks or a self-contained prompt you hand to that repo's own agent. Reading for context is fine.
3. **Verdicts are designer-only.** Agents produce evidence (proofs, batteries, reports); they never produce approval.
4. **Pins are deliberate.** A consumer holding an older upstream version is a recorded choice; check the pin's doc before touching it.

When an agent flags a conflict: that is the system working. Stop, take it to planning, rule it there, let the ruling flow back down.

4. The decision system

4.1 The Decision Deck

`planning tools/decision_deck.html` — offline single-file app, your build, adopted as THE decision register. The committed `decision_deck_register.json` is the record; the app is the surface.

Six card types: **Ruling** (one-tap choice, your lean highlighted), **Feel verdict** (play and judge; `fresh` tier hides until you flip FRESH HANDS on), **Play evidence** (do once, logs prove it — tired counts), **Acceptance row** (formal sign-off from captures/reports), **Go/no-go** (one DO IT authorizes an engineering batch), **Cadence** (recurring).

Your loop: PASTE FROM AI (append cards a session staged) → deal → decide, optional note → EXPORT JSON → hand it to the next session, which sweeps decisions into the register and session log. Card text obeys the plain-language rule — if a card reads as jargon, that's a defect; bounce it back.

4.2 Two-tier verdicts (the fresh-hands rule)

- **Tier 1:** your in-session calls — chat one-liners, deck taps — are decisions **immediately**. No ceremony.
- **Tier 2:** *feel* items additionally get ONE rested ratification pass before they are final.
- Dodgeability verdicts accept {rested-human, bot-proof}. **Feel verdicts accept rested humans only — never a bot.** Bots verify mechanics, never feel.

4.3 "Rested" — the honest definition

Keys on hours into **your waking day and session**, never wall clock. Your shift is 15:00–23:00, so home at midnight is your ~17:00 — not rested. Rested = day start, fresh, on a clean build. Marathon sessions and dirty runs (god, slow-mo, runtime edits) stay PROVISIONAL regardless of tier. Any runtime edit auto-stamps subsequent feel notes PROVISIONAL — the stamp is honored, never overridden by enthusiasm at hour 14.

4.4 Recording verdicts in-game

Console (` key):

```
verdict <dodgeability|feel> <rested-human|bot-proof> <text>
```

Feel rejects bot sources by design. Verdicts land in `user://logs/verdicts.jsonl`; the next session sweeps them into the planning log. Record feel on clean runs only.

4.5 Talk-before-build (LAW)

For NEW protocol/tool/system ideas: the design conversation happens in chat FIRST — the agent builds nothing until you have walked the idea through end-to-end and said build. It does not slow normal in-lane engineering you already authorized. (Origin: the sync protocol's first draft got built in one pass the moment it was mentioned; you called it out; the rule makes the process mandatory.)

5. Session protocol

5.1 Which repo do I open? (the recipe)

1. Open the repo where the changes will land. **One repo per session.**
2. Planning only needs to exist on disk. Exceptions: game sessions keep the game + planning pair; planning/decision sessions start in planning.
3. Cross-repo needs → recorded asks or ready-to-paste prompts for that repo's own agent. Never work done out-of-lane.

5.2 Session start (the agent's ritual — know it to check it)

The agent must: read the contract docs; verify the checkout is on the ruled mainline (or designated branch) and not behind remote — a stale or wrong-lane checkout is a STOP with an incident entry; skim the sync log for open entries touching the repo; confirm lock-file pins match disk if the repo consumes packs. If it skipped these, tell it to do them.

Your part: `hourslog start`, state the goal, answer Tier 1 questions as they come.

5.3 Session end

The agent must: update HANDOFF; append sync-log entries for every cross-repo event the session caused (**no event, no entry** — the log is events, not diary); commit, push, and **verify the push landed** via `git status -sb`. Your part: sweep the deck if cards were staged or tapped, then `hourslog stop`.

5.4 Janitor sessions (git hygiene only)

A planning-rooted session may do cross-repo git housekeeping under five mandatory guardrails: **(a)** archive tags pushed before anything moves; **(b)** that repo's gates green before any push — not green or not runnable means no push without your explicit word; **(c)** mechanical only — conflicts, verdict salvage, and "which line wins" escalate to you; **(d)** live-session check before touching any branch; **(e)** destructive ref operations (branch deletion, force-push) are **executed by you**, never the agent — it stages the exact command, you click.

6. Cross-repo logistics

6.1 One home per kind of information

Information	The ONE home
Decisions / rulings / sign-offs	Decision Deck register
Daily narrative	`planning notes/sessions/<date>.md`

Information	The ONE home
Per-repo current state	that repo's HANDOFF.md
Cross-repo events	`planning tools/sync_log.json`
Version pins	`planning tools/ecosystem.lock.json`
Tech debt	the owning repo's ledger
Mechanical evidence	the producing repo's `reports/` (linked, never copied)

Cross-references by path/id, never by copying content.

6.2 The sync log — your triage surface

Event types: delivery, intake, ask_opened, ask_resolved, pin_change, incident, handoff, ruling, note. Entry titles obey the plain-language rule because **you** read them.

`tools/sync_log.html` top board computes three lists from the entries — **deliveries waiting for intake** (exported, nobody took it in yet), **open problems**, **open asks**. Your one action: mark resolved with a one-line note → EXPORT → commit (or hand to the next session). A delivery with no matching intake is by definition *waiting* — this retires "someone remembers to carry the directory over."

6.3 Deliveries (producer side)

Publish gates refuse to export from a dirty tree or an unpushed HEAD. Manifests carry sourceCommit + hashes. The pack zip ships as a **GitHub release asset** tagged with the artifact id — the registry replaced desktop-zip/chat transport by your ruling. Delivery gets a sync-log entry with id, sourceCommit, hash, pickup location.

6.4 Intakes (consumer side — today: the game)

Validators + battery + runbook run first; the manifest's sourceCommit/hash is verified against the delivery entry. **Mismatch = incident + STOP — never intake an artifact whose origin can't be confirmed.** Then the lock file's pin updates and an intake entry references the delivery. You never bypass this, even for your own drops.

6.5 Asks and incidents

Needs flow upstream as `ask_opened` entries (plus an `upstream-ask` GitHub issue on the target repo); resolution logs `ask_resolved`. Anything violating the cross-repo rules — drift, unverifiable artifact, stale clone, hand-edited export — gets an `incident` at discovery time. Incidents are facts, not blame; they exist so the third repetition of a failure mode is impossible, not embarrassing.

7. Playtesting, evidence, and verdicts

7.1 Quiet-lab law

Gate sessions are 2–3 warm **watched** first-touches (screen-share or in person), scheduled by you only once the loop bar holds. Watching is **silent** — no coaching, no prompting, no explaining during the run. The gate is never judged solely by the builder.

7.2 CORE-54 evidence rules

Only **unprompted** game-descriptions count — harvested verbatim from Discord/itch/feedback channels into the CORE-54 log. Any debrief answer to a direct question is marked "prompted". The in-build comments box is supplementary and never logged as CORE-54 evidence.

7.3 The feedback return path

Testers: options menu → "feedback: save bundle" → zip lands on their Desktop with a HUD toast showing path + summary code (`WS1-MMM-SSDD-KKKC`, checksummed — it refuses typos). Your tools: `tools/decode_summary_code.py` (decode a pasted code) and `tools/evidence_report.py <bundle.zip|session.jsonl>` (per-session durations, loadout speeds, gaps, contamination scan — measurements only; interpretation stays yours). Dev-profile sessions are excluded from codes automatically.

7.4 Acceptance work you sign

- **9-row acceptance** (readability laws): eye rows judged from the committed density-audit captures, including the deutan capture; sign-off table in `game notes/NINE_ROW_ACCEPTANCE.md`.
- **Eyes-closed audio** (Law 7): one mixed encounter, eyes closed — can you tell the cue classes apart? Verdict + date in `game notes/AUDIO_CUE_MAP.md`. Now runnable against real Resonance Forge cues.
- **CORE-50 render half:** the mechanized half is proven by gates; the render half is one tester-build pass with the checklist in `game notes/CORE50_RUNTIME_CHECKLIST.md`.

8. The Loop milestone (the current target)

The bar: an unguided complete run — spawn in the b65 town, walk out, fight through rising danger where loot actually drops and matters, reach the first boss or die trying, death costs something real, dying pulls you to retry immediately — that stays **fun for YOU playing it daily for a week**. Feel and stakes over polish. The bar's final wording is yours (deck card staged).

Your protocol:

1. Look at the L1 skeleton (creation screen, drops, HUD, THE LOOP picker row) and call it **judgeable** — that starts the bar clock.
2. Play THE RUN daily. Tier 1 notes as you go; loop tuning (L2) flows from what you say. Every [T] rate is yours to confirm or change: drop tables, XP growth, the 25% death gold cost, king at 900 HP, unique at 35%, tint ladder.
3. Daily play throws off weekly GIF material for free.
4. Bar holds for a week → schedule the warm watched first-touches. The export pipeline + lockdown profile are a standing gate so clean-stamped tester zips are always ready.

Anything outside the loop bar's needs gets refused and ledgered or flagged to planning — the scope tripwire is protection, not bureaucracy.

9. Production duties (PROD-01 and cadence)

- **Hours:** `tools/hourslog.ps1 start` before ANY project work — code, art, design, planning, deck sessions on the bus. `stop` after. Dev hours ≠ game-running hours; everything is logged.
- **The floor:** `tools/hours_report.ps1` weekly; a 4-week rolling average under 40 h/week triggers the floor reset, the slip ladder, and roadmap re-derivation — by rule, not mood. The rule fires automatically; your job is only to not argue with it.
- **Weekly GIF:** one 30–60 s GIF per week (`G` → `tools/gif.ps1`), posted to the devlog + one community. A deliverable, not garnish — it primes the tester pipeline the warm touches draw from. Never skips.
- **Tester builds:** `tools/export.ps1` cuts dev + tester zips, boot- checked by exit code, butler command prepared for you. The lockdown lint + probe guarantee tester zips contain no dev tools. The pretester checklist (`tools/pretester_check.ps1`) is the one-command gate — it needs **exclusive Godot access** (close your editor/game).

10. Per-repo operator notes and landmines

- **Game:** never edit sim data while a battery runs. Gates read exit codes, not prose. Dev keyboard has no F-row — debug keys stay on letters/punctuation. Speed presets are tester-facing; god/slow-mo are dev-only and dirty the replay.
- **Planning:** the deck register and sync log are committed JSON — export from the apps and commit; localStorage is per-browser, not the record.
- **WorldForge:** WYSIWYG is the ruled collision doctrine (art outline minus declared pass cells). Porosity pins are per-drop. Your walks are the acceptance.
- **world_filler:** mainline is `main` (your approval line); format 1 FINAL is the consumption basis; game-side consumption is post-Gate-1 by doc 17. Node ≥ 24.15; the canonical 256² pack is regenerated, not committed.
- **tileforge:** feature-complete, consumption phase; gates run locally only. REF3 v2 promotion is yours to gate.
- **Assembler:** the pack contract is planning docs/14 — binding. Boss pack stays a raw drop until a boss sprite is naturally needed; polished re-exports get revalidated with the tool before replacing the directory.
- **Resonance Forge:** the taste profile is fragile (138 BPM, F mixolydian, no vibrato, sparse fragmented lead — 9 iterations to find; do-not-regress). Automated checks may REJECT audio but never approve it — your listening gate is mandatory. Env: don't edit store modules while Vite runs.

11. Red lines (the never-do list)

1. Never resolve a design conflict in an implementing repo.
2. Never let an agent do another repo's work in-lane ("it's quicker" is how lanes die).
3. Never accept a feel verdict from a bot, or from your own tired hands.
4. Never override a PROVISIONAL stamp because it felt great.
5. Never hand-edit an exported artifact or hand-carry a pack.
6. Never intake an artifact whose origin can't be hash-verified.
7. Never upgrade a pin without a ruling.
8. Never coach, prompt, or explain during a watched run.
9. Never let an agent delete branches or force-push — you click.
10. Never build a new system idea without the talk-first conversation.
11. Never skip the weekly GIF or the hours log.
12. Never run determinism gates with another Godot instance holding the

```
project open, and never edit sim data mid-battery.
```

12. Glossary (plain language)

- **Tier 1 / Tier 2** — decision now / feel decision that also needs one rested confirmation pass.
- **[L] / [P] / [T] / [CUT]** — locked / provisional / test-gated (a number playtesting may change) / deliberately removed.
- **Battery** — the full set of bot proof runs; its reports are the record. "Byte-identical" means nothing in the sim changed.
- **Goldens** — reference replays the build must reproduce exactly; regenerating them is deliberate, never incidental.
- **Pretester** — the one-command gate that must be all-green before a tester build ships.
- **Delivery / intake** — a pack shipped by a producer / verified and adopted by a consumer. A delivery without an intake is *waiting*.

- **Pin** — the exact upstream version a consumer holds, on purpose.
- **Canary** — a proof designed to fail (or trivially pass) to prove the proof system itself still works.
- **Dirty run** — god mode, slow-mo, or runtime edits; feel notes from it are PROVISIONAL.
- **CORE-54 log** — unprompted player descriptions of the game, verbatim.
- **The bar / loop bar** — THE LOOP staying fun for you, daily, for a week.
- **Janitor session** — planning-rooted git housekeeping under the five guardrails.

13. Where everything lives

Need	Path
Ecosystem map	`planning docs/16-ECOSYSTEM_MAP.md`
Sync protocol	`planning docs/18-AGENT_SYNC_PROTOCOL.md`
Decision register (full text)	`planning docs/08-DECISION_REGISTER.md`
Deck app + register	`planning tools/decision_deck.html` / `decision_deck_register.json`
Sync log app + record	`planning tools/sync_log.html` / `sync_log.json`
Version pins	`planning tools/ecosystem.lock.json`
Loop spec	`planning docs/19-LOOP_MILESTONE_SPEC.md`
Game contract	`game CLAUDE.md`
Pack intake runbook	`game notes/PACK_INTAKE_RUNBOOK.md`
Tech debt ledger	`game notes/TECH_DEBT_LEDGER.md`
Daily truth	`planning notes/sessions/<date>.md`